1. Introduction

Schools need a steady way to organize examination marks and make results easier to review. When marks are handled only on paper, staff may find it difficult to update a record, locate a result, or check whether a mark has been entered correctly. This project presents a small Digital Exam and Results Management System written in Python. It is an educational demonstration of Object-Oriented Programming (OOP), rather than a complete output system.

2. Problem Statement

The project addresses the main organization of student examination records. The program represents students and examinations, records marks against the correct student and subject, calculates grades, and show results. It also checks marks and prevents duplicate student or examination tags.

3. Aim and Objectives

The aim is to develop a Python program that applies OOP concepts to a practical problem in education. The objectives are to:

- Create classes for students, examinations, and results management.
- Use attributes to store relevant student and examination information.
- Execute methods for updating records, verifying marks, calculating grades, and displaying results.
- Demonstrate object creation and interaction between objects.
- Consider open-source sharing, accessibility, privacy, and division.

4. System Design

4.1 Student class

The Student class represents one student. Its attributes are student_id, name, course, and results. The update_record() method updates the student's name or course. The display_info() method displays the student's details, while add_result() stores a mark together with the related Exam object.

4.2 Exam class

The Exam class represents an examination and stores an exam ID, subject, and highest mark. The approve_mark() method checks whether a mark is within the permitted range. The calculate_grade() method converts a valid mark into a grade using the clear boundaries in this project.

4.3 ResultsManagementSystem class

This class coordinates the other objects. It maintains lists of Student and Exam objects, checks for duplicate IDs, finds records, records marks, and displays results. Separating these responsibilities makes the program easier to understand and expand.

5. OOP Concepts Used

Concept	Meaning in this project	Example
Class	Blueprint for creating objects	Student, Exam
Object	An instance of a class	student1, exam1
Attributes	Information held by an object	student_id, subject, mark
Instance methods	Work with an object	update_record(), display_info()
Class method	Changes class-level information	change_school_name()
Static method	A utility check that needs no object state	valid_student_id()
List	Stores multiple objects	students and exams lists

Object interaction occurs when record_result() finds a Student object and an Exam object, then asks the student to store the mark for that examination. The result keeps a reference to the Exam object, so the subject name and highest mark are available when the result is displayed.

6. How the Program Works

1. A Results Management System object is created.
2. Two Student objects and two Exam objects are created using imaginary information.
3. The objects are registered with the management system.
4. Marks are recorded using a student ID, examination ID, and mark.
5. The program displays the student's details, subject, mark, and calculated grade.
6. A student's course and an existing examination mark are updated.
7. An invalid mark is tested and rejected.

7. Sample Results and Testing

The program was run using imaginary student records. The table summarises the key tests. The grade boundaries are helpful for this assignment and are not presented as an official WAEC or institutional grading scale.

Test	Input/action	Observed behaviour
Register students	ST001 and ST002	Student records added
Add examinations	EX001 Mathematics; EX002 Programming	Exams added
Record a mark	ST001, EX001, 78	Mark saved; grade B
Calculate a grade	ST001, EX002, 86	Grade A
Update a result	ST001 Mathematics changed to 82	Existing result updated; grade A
Validate a mark	Enter 120 out of 100	Invalid mark rejected

8. Digital Public Goods (DPG) Principles

8.1 Open-source

The source code can be published in a public GitHub repository with an appropriate open-source licence. Other learners and developers can inspect, reuse, and adapt the project under the licence terms.

8.3 Privacy-respecting

The demonstration uses fictional information and does not request passwords, home addresses, medical information, or other sensitive details. Records are kept in memory and disappear when the program closes. A real deployment would need appropriate access controls and a proper data-protection process.

8.4 Modular and reusable

Student information, examination information, and management operations are separated into classes. This makes the code easier to maintain and provides a basis for later additions such as file storage, a database, or an institution-approved grading scale.

9. Limitations and Future Improvements

This version is a command-line demonstration. It does not save records after exit, authenticate users, provide a graphical interface, or generate official transcripts. Future improvements could include persistent storage, authorized user roles, printable reports, additional input validation, and configurable grade boundaries.

10. Conclusion

The Digital Exam and Results Management System shows how Python OOP can be applied to an education-related problem. The Student, Exam, and ResultsManagementSystem classes have separate responsibilities and work together to record, update, and display examination results. The project also considers basic validation, accessibility, privacy, open-source sharing, and modular design.

Python program output - registration and initial results

```
DIGITAL EXAM AND RESULTS MANAGEMENT SYSTEM
==========================================
Student ST001 registered successfully.
Student ST002 registered successfully.
Exam EX001 added successfully.
Exam EX002 added successfully.
Result saved: Mariama Kamara - Mathematics: 78/100.
Result saved: Mariama Kamara - Programming: 86/100.
Result saved: Abdul Sesay - Mathematics: 65/100.

========== STUDENT RESULT ==========
Student ID: ST001
Name: Mariama Kamara
Course: Computer Science
School: Community Learning Centre
-----------------------------------
Subject: Mathematics
Mark: 78/100
```

Figure 1. Registration and initial examination results.

Python program output - update and validation

```
Grade: B
-----------------------------------
Subject: Programming
Mark: 86/100
Grade: A
===================================

========== STUDENT RESULT ==========
Student ID: ST002
Name: Abdul Sesay
Course: Information Technology
School: Community Learning Centre
-----------------------------------
Subject: Mathematics
Mark: 65/100
Grade: C
===================================

--- Updating an existing record ---
Result saved: Mariama Kamara - Mathematics: 82/100.

========== STUDENT RESULT ==========
Student ID: ST001
Name: Mariama Kamara
Course: Software Engineering
School: Community Learning Centre
-----------------------------------
Subject: Mathematics
Mark: 82/100
Grade: A
-----------------------------------
Subject: Programming
Mark: 86/100
Grade: A
===================================

--- Validation check ---
Error: Mark must be within the permitted range.

Demonstration completed successfully.
```

Figure 2. Updated record and invalid-mark validation.